

부록 C. 프로젝트 아키텍처 지도

이 부록은 새 개념을 설명하지 않는다. Book-forge 소스 전체(약 58개 실질 파일)가 어떤 패키지로 나뉘고, 각 파일이 무엇을 책임지며, 어느 챕터에서 다뤄지는지 한자리에 모아둔 **참조용 지도**다. 처음부터 끝까지 읽을 필요는 없다. 본문을 읽다가 낯선 파일 이름이 나올 때 돌아와 찾아보는 용도로 쓰면 된다.

C.1 프로젝트 아키텍처 — 패키지 구조와 의존 라이브러리

Book-forge의 실제 코드는 `src/book_forge/` 아래 7개 서브패키지(+ 루트 3개 모듈)로 나뉜다. 이 절은 "어떤 묶음이 무슨 일을 하는가"를 30,000피트 상공에서 조망한다. 각 파일의 역할은 바로 다음 절(C.2)에서 표로 정리한다.

```
src/book_forge/
├─ models.py, config.py, exceptions.py  # 루트 - 목차 데이터 모델·설정·예외 계층(3개 파일, 특정 패키지에 속하지 않음)
├─ llm/                                # LLM Protocol + OpenAI·Anthropic·Ollama 통합(provider.py 1개 파일)
├─ agents/                             # LLM 에이전트 13개 + @tool_guard 1개(scaffold) + 정적 검증기 5개 + 프롬프트 상수 1개
├─ knowledge/                          # RAG - 소스 어댑터·임베딩·지식창고·구조적 코드 인덱싱(7개 파일)
├─ publish/                            # 마크다운 → HTML/PDF/EPUB/Slides(9개 파일)
├─ editor/                             # Flask 웹 에디터(server.py 1개 파일)
├─ eval/                               # PerformanceMonitor 팩토리(2개 파일)
└─ cli/                                # click 진입점 + 13개 서브커맨드(15개 파일)
```

`__init__.py` 처럼 실질적인 로직이 없는 빈 파일을 제외하면 약 58개 파일, 총 7,019 줄(조사 시점 `wc -l` 기준)이 Book-forge 전체 구현이다. 이 책이 지금까지 인용해온 코드는 전부 이 58개 파일 중 일부다.

의존 라이브러리는 `pyproject.toml` 이 코어와 옵트인 extra를 명확히 나눈다:

구분	라이브러리	어디서 쓰는가
코어(항상 설치)	<code>click</code>	<code>cli/</code> 전체 — 커맨드 정의
코어	<code>python-dotenv</code>	<code>config.py</code> — <code>.env</code> 로드
코어	<code>pydantic</code>	(설정 검증용으로 선언돼 있으나 이 책이 다루는 핵심 경로에서는 직접 노출되지 않는다)
코어	<code>markdown</code>	<code>publish/markdown_engine.py</code> — <code>md_to_html()</code> 의 실제 변환 엔진
코어	<code>requests</code>	<code>knowledge/embeddings.py</code> (Ollama API), <code>knowledge/web_search.py</code> , <code>knowledge/sources.py</code> (URL 소스)
코어	<code>agent-evaluator ≥ 0.9.9</code>	<code>agents/*</code> (<code>@agent_eval</code> / <code>@tool_guard</code>), <code>eval/monitor.py</code> , <code>cli/commands/gate_cmd.py</code>
[pdf]	<code>playwright</code>	<code>publish/pdf_builder.py</code> — Chromium headless 인쇄
[rag]	<code>pypdf</code> , <code>numpy</code>	<code>knowledge/pdf_source.py</code> (PDF 추출), <code>knowledge/store.py</code> (코사인 유사도 행렬 연산)
[serve]	<code>flask</code>	<code>editor/server.py</code>
[korean]	<code>agent-evaluator[korean]</code>	<code>eval/monitor.py</code> 의 <code>use_korean_tokenizer=True</code> 가 기대하는 형태소 분석기

⚠ **pyproject.toml 에 없는 의존성:** `llm/provider.py`는 provider로 OpenAI 또는 Anthropic을 선택했을 때만 `from openai import OpenAI` / `from anthropic import Anthropic`을 함수 내부에서 지연 import한다. 두 SDK 모두 `pyproject.toml`의 어떤 의존성 목록에도 선언돼 있지 않다. 기본값인 Ollama만 쓰면 이 문제를 겪을 일이 없지만, `LLM_PROVIDER=openai`로 전환하려면 `pip install openai`를 별도로 실행해야 한다. 이 책이 반복해서 강조하는 "관대한 파싱, 그러나 숨겨진 전제조건은 정직하게 문서화한다"는 원칙이 놓치기 쉬운 지점이 바로 여기서.

이 8개 패키지 전부가 공통으로 따르는 두 가지 배선 패턴(`build_X(llm, monitor)` -> `Fn` 팩토리, `@agent_eval` vs `@tool_guard` 의 축 구분)은 이미 이 책의 핵심 주제이므로 2·8·14장에서 각각 코드로 따라간다. 이 절은 "그 패턴들이 정확히 어느 파일에 있는가"의 지도 역할만 한다.

C.2 파일별 책임 지도

아래 표는 Book-forge 소스 전체(58개 실질 파일)를 패키지별로 나눠, 각 파일의 책임과 이 책에서 주로 다루는 곳을 정리한 것이다. 처음 읽을 때 전부 외울 필요는 없다.

루트 모듈

파일	책임	핵심 요소
<code>models.py</code>	목차 데이터 모델 — <code>ChapterSpec</code> , <code>toc</code> 매니페스트 파싱, 목차 개정 이력 조작	<code>ChapterSpec</code> , <code>parse_toc_manifest()</code> , <code>append_toc_revision_entries()</code>
<code>config.py</code>	<code>~/Documents/BookForge/</code> 데이터 디렉토리 관리, <code>.env</code> 경로 해석	<code>get_data_dir()</code> , <code>load_config()</code> , <code>project_dir_for()</code>
<code>exceptions.py</code>	예외 계층	<code>BookForgeError</code> (base), <code>MissingAPIKeyError</code> , <code>LLMProviderError</code> , <code>TocParseError</code> 등

llm/ — LLM 통합

파일	책임	핵심 요소
<code>llm/provider.py</code>	OpenAI/Anthropic/Ollama를 하나의 인터페이스로 통합, provider별 SDK 지연 로딩, 빈 응답을 예외로 전파	<code>LLM</code> (Protocol), <code>OpenAILLM</code> / <code>AnthropicLLM</code> / <code>OllamaLLM</code> , <code>create_llm()</code> , <code>_require_non_empty()</code>

`agents/` — LLM 호출 에이전트(13개, `build_X(llm, monitor)` → `Fn` 팩토리 + `@agent_eval`)

파일	책임	담당 챕터
<code>agents/planner.py</code>	PlannerAgent — 주제/제약 → 기획안 마크다운	2장
<code>agents/toc_designer.py</code>	TOCDesignerAgent — 기획안(+코드 구조) → 목차 마크다운	4장
<code>agents/chapter_drafter.py</code>	ChapterDrafterAgent — narrative/exercise 서술형 챕터 초안(기본 생성기)	7장
<code>agents/reference_table.py</code>	ReferenceTableAgent — RAG 소스에서 구조화된 사실만 표로 추출	7장
<code>agents/diagram_generator.py</code>	DiagramGeneratorAgent — RAG 소스 → Mermaid 다이어그램 중심 챕터	7장
<code>agents/capstone_generator.py</code>	CapstoneGeneratorAgent — 빈 템플릿+모범 정답을 한 번의 호출로 생성	7장
<code>agents/module_reference.py</code>	ModuleReferenceAgent — 구조적 코드 인덱싱 결과를 빠짐없이 표로 정리(전체 커버리지)	7장
<code>agents/alternative_suggester.py</code>	AlternativeSuggesterAgent — 저커버리지 상황에서 자동 차단 대신 대안 제시	3장
<code>agents/chat_agent.py</code>	ChatAgent — 지식창고 발췌+대화 이력 기반 Q&A	7장
<code>agents/research_agent.py</code>	ResearchAgent — 챕터 제목 → 검색 쿼리 생성(검색 자체는 <code>knowledge/web_search.py</code>)	7장
<code>agents/review_loop.py</code>	AuthorReviewLoop — 저자 피드백에 따른 반복 개정 (<code>run_review_loop()</code> 는 LLM 미호출 순수 오케스트레이션)	6장
<code>agents/review_panel.py</code>	ReviewerAgent(정확성/가독성)·ChiefEditorAgent — 감독자·작업자 다관점 리뷰	5장
<code>agents/slide_condenser.py</code>	SlideCondenserAgent — 책 섹션 하나 → 슬라이드 한 장	(범위 밖)

agents/ — 그 외(LLM 호출 방식이 다르거나 아예 안 하는 7개 파일)

파일	책임	비고
<code>agents/scaffold.py</code>	ScaffoldAgent — 승인된 목차 → 빈 챗터 스텝 생성 + 목차 재조정	<code>@agent_eval</code> 이 아니라 <code>@tool_guard</code> (부작용 있는 파일 쓰기, 실행 전 차단 대상) — 14장
<code>agents/prompts.py</code>	위 13개 에이전트 중 다수가 공유하는 프롬프트 템플릿 문자열 저장소	순수 상수 모듈, LLM 미호출
<code>agents/demonstration_verifier.py</code>	content_type별 생성 후 정적 검증 (exercise 문법·diagram 구조·capstone TODO 등)	LLM 미호출, <code>@agent_eval</code> 없음 — 11장
<code>agents/code_consistency_checker.py</code>	본문의 import/백틱 심볼이 실제 target_package에 존재하는지 대조	LLM 미호출 — 11장
<code>agents/sdk_version_pin.py</code>	<code>--check-package</code> 대상 버전을 프로젝트별로 최초 1회 고정, 드리프트 경고	LLM 미호출 — 11장
<code>agents/code_examples_verifier.py</code>	python 코드 블록을 subprocess로 실제 실행해 exit code 검증(<code>--execute-examples</code>)	LLM 미호출 — 11장
<code>agents/term_consistency_checker.py</code>	챗터 간 백틱 기술 용어 표기 불일치 후보 검출	LLM 미호출 — 11장

knowledge/ — RAG

파일	책임	담당 챗터
<code>knowledge/store.py</code>	<code>KnowledgeStore</code> — numpy 인메모리 코사인 유사도 벡터 스토어, JSON 영속화	7장
<code>knowledge/embeddings.py</code>	Ollama <code>/api/embeddings</code> 호출 래퍼(RAG는 항상 Ollama만 사용)	7장
<code>knowledge/sources.py</code>	PDF/코드저장소/텍스트/URL을 동일한 청크 리스트로 변환하는 소스 어댑터 라우터	7장

파일	책임	담당 챗터
<code>knowledge/pdf_source.py</code>	PDF → 텍스트 → 고정폭 슬라이딩 청크	7장
<code>knowledge/web_search.py</code>	DuckDuckGo HTML 엔드포인트 검색(API 키 불필요)	7장
<code>knowledge/code_index.py</code>	Python <code>ast</code> 로 모듈/클래스/함수 인벤토리 + 의존관계 정적 분석	4·7장
<code>knowledge/lifecycle.py</code>	지식창고 청크를 소스 태그별로 집계(<code>book-forge knowledge status</code>)	(범위 밖)

publish/ — 빌드

파일	책임	비고
<code>publish/config.py</code>	<code>BookConfig</code> — HTML/PDF/EPUB/Slide 엔진 공통 설정 dataclass	이 책 범위 밖 (README 참고)
<code>publish/front_matter.py</code>	<code>FrontMatter</code> — 저자/저작권/판 메타데이터를 별도 JSON으로 분리 저장	//
<code>publish/toc_loader.py</code>	<code>01_목차.md</code> 매니페스트 → 실제 파일 경로가 채워진 <code>ResolvedChapter</code> 목록	4장
<code>publish/book_index.py</code>	책 전체 찾아보기(색인) 항목 빌드 (<code>term_consistency_checker</code> 의 용어 추출 재사용)	//
<code>publish/markdown_engine.py</code>	<code>md_to_html()</code> — mermaid/커스텀 HTML/코드 보존 3 단계 마크다운 변환 엔진(공용)	//
<code>publish/html_builder.py</code>	전체 도서를 단일 HTML 파일로 빌드(사이드바 내비게이션 자동 생성)	//
<code>publish/pdf_builder.py</code>	Playwright(Chromium headless)로 챗터별 개별 PDF 인쇄	//
<code>publish/epub_builder.py</code>	순수 <code>zipfile</code> (표준 라이브러리)로 EPUB 3 컨테이너 조립	//

파일	책임	비고
<code>publish/slide_builder.py</code>	마크다운 챗터 → Reveal.js 발표자료 (<code>slide_condenser.py</code> 호출)	//

editor/ , eval/

파일	책임	담당 챗터
<code>editor/server.py</code>	Flask 웹 에디터 — Part/Chapter MD 편집 + 팀 동시성 클레임 충돌 방지	15장
<code>eval/monitor.py</code>	<code>PerformanceMonitor</code> 팩토리 — 한국어 형태소 토크나이저, Gate 가중치 <code>.env</code> 오버라이드	16장
<code>eval/gate_summary.py</code>	방금 저장된 평가 결과 JSON에서 Gate A-G 점수를 즉시 읽어 CLI에 표시	9장

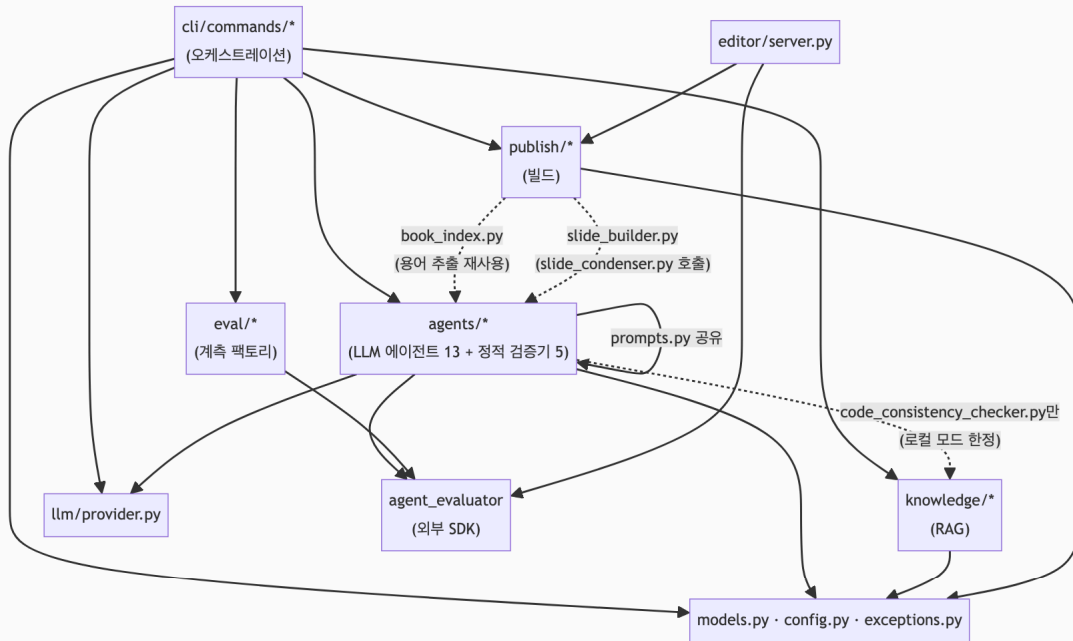
cli/ — 명령 오케스트레이션

파일	책임	담당 챗터
<code>cli/main.py</code>	click 그룹 진입점 — 13개 서브커맨드 등록	1장 § 1.2
<code>cli/project_utils.py</code>	슬러그 → <code>BookConfig</code> 해석 공통 유틸 (build/edit/gate/draft/plan이 공유)	(범위 밖)
<code>cli/commands/new_cmd.py</code>	<code>book-forge new</code> — 기획~스캐폴딩 오케스트레이션, <code>--source</code> 배치 초안 연쇄	4장
<code>cli/commands/draft_cmd.py</code>	<code>book-forge draft</code> — RAG 초안 생성 오케스트레이션(733줄, 가장 큰 파일)	7장
<code>cli/commands/gate_cmd.py</code>	<code>book-forge gate</code> — 여러 챗터 결과 병합 + <code>agent-eval gate</code> CLI 위임 + CI 연동 플러그	9·12·13장
<code>cli/commands/plan_cmd.py</code>	<code>book-forge plan</code> — 기획/목차 재검토, <code>--revise</code> 재승인 루프	6장

파일	책임	담당 챗터
<code>cli/commands/review_cmd.py</code>	<code>book-forge review</code> — 리뷰 패널 호출 래퍼	5장
<code>cli/commands/chat_cmd.py</code>	<code>book-forge chat</code> — 지식창고 기반 대화형 Q&A REPL	7장
<code>cli/commands/research_cmd.py</code>	<code>book-forge research</code> — 검색 쿼리 생성 → 웹 검색 → 저자 선택 → 지식창고 추가	7장
<code>cli/commands/lint_cmd.py</code>	<code>book-forge lint</code> — 챗터 간 용어 불일치 발견·보고	11장
<code>cli/commands/build_cmd.py</code>	<code>book-forge build html\ pdf\ epub\ slides</code> 서버그룹	(범위 밖)
<code>cli/commands/edit_cmd.py</code>	<code>book-forge edit</code> — 웹 에디터 서버 실행	15장
<code>cli/commands/init_cmd.py</code>	<code>book-forge init</code> — LLM provider/API 키 대화형 <code>.env</code> 설정 마법사	(범위 밖)
<code>cli/commands/home_cmd.py</code>	<code>book-forge home</code> — 데이터/프로젝트 폴더를 OS 파일 탐색기로 열기	(범위 밖)
<code>cli/commands/knowledge_cmd.py</code>	<code>book-forge knowledge status\ reset</code>	(범위 밖)

C.3 모듈 간 관계 — 누가 누구를 부르는가

파일 수십 개 전부를 하나의 그래프에 그리면 알아볼 수 없으므로, 패키지 단위로 뭉쳐 실제 import 관계를 그린다. 화살표는 전부 실제 `from book_forge.X import Y` 문을 근거로 삼았다.



이 그래프에서 눈에 띄는 비대칭이 두 가지 있다.

1. **agents/** 는 **knowledge/** 를 몰라도 대부분 동작한다. RAG 소스(**sources**)는 **cli/commands/draft_cmd.py** 가 **knowledge/store.py** 에서 미리 꺼내 문자열로 넘겨주므로, 대부분의 에이전트 함수는 **KnowledgeStore** 객체 자체를 본 적이 없다. 유일한 예외는 **agents/code_consistency_checker.py** (점선 화살표)다. 이 파일만 로컬 디렉토리 모드에서 **knowledge/code_index.py** 를 직접 지연 import한다.
2. **publish/** 는 원칙적으로 **agents/** 를 몰라야 하는 레이어지만(빌드는 집필과 무관한 별도 단계), 실제로는 두 파일이 예외다. **publish/book_index.py** 는 **agents/term_consistency_checker.py** 의 용어 추출 함수(순수 함수, LLM 미호출)를 재사용한다. 찾아보기(색인)를 만들 때 "이미 있는 로직을 새로 만들지 않는다"고 판단해 생긴 얇은 예외다. **publish/slide_builder.py** 는 이보다 근본적으로 의존한다. 챕터 섹션을 슬라이드 한 장으로 압축하는 작업 자체를 **agents/slide_condenser.py** 의 **build_condense_section()** (LLM 호출 에이전트)에 위임하기 때문이다. 그래서 "빌드는 집필과 무관하다"는 원칙이 발표자료 빌드에는 처음부터 적용되지 않는다.

editor/server.py 가 **publish/** (마크다운 렌더링)와 **agent_evaluator** (팀 동시성 가드레일) 둘만 직접 의존하고 **agents/** 나 **knowledge/** 는 전혀 모른다는 점도 이

그래프에서 확인할 수 있다. 웹 에디터는 "사람이 직접 쓴 텍스트를 저장·미리보기"할 뿐, LLM을 한 번도 호출하지 않는다.

참고 자료

- [Book-forge/README.md](#) — CLI 전체 옵션과 설치 방법
- [Book-forge/CLAUDE.md](#) — 프로세스 아키텍처 전체 다이어그램과 파일 구조
- 1장(§ 1.1~ § 1.3) — Book-forge 파이프라인 전체 그림과 CLI 명령 표(이 부록보다 앞서, 개념 위주로 다룸)